GUIDE: QR Watchman Attendance System

This guide provides the essential steps to set up, run, and deploy the QR Watchman Attendance System. Follow these instructions to get your project working locally and in production.

Prerequisites

- .NET 8 SDK
- Node.js 18+ and npm
- SQL Server or SQL Server Express
- Google Maps API key
- *(optional)* Azure account for deployment
- *(optional)* Cloudflare account for Pages/Workers

Quick Start: Local Development

1. Clone the repository

```
git clone https://github.com/your-username/QR-Watchman-Attendance-System.git
cd QR-Watchman-Attendance-System
```

2. Set up the database

- Create an empty SQL Server database.
- Run the setup script:

```
sqlcmd -S localhost -d WatchmanAttendance -i sql/setup.sql
```

- Or open `sql/setup.sql` in SQL Server Management Studio and execute it.
- **Default admin login:**
  - Username: `admin@surge.com`

- Password: `12345678`
- **Change the admin password after first login!**

3. Configure the backend

- Edit `WatchmanAttendance.API/appsettings.json`:
  - Set your database connection string and JWT secret.
  - Example:

```
{
  "ConnectionStrings": {
    "Default": "Server=localhost;Database=WatchmanAttendance;Trusted_Connection=True;"
  },
  "Jwt": {
    "Key": "your-secret-key-at-least-32-characters",
    "Issuer": "WatchmanAttendanceAPI",
    "Audience": "WatchmanAttendanceClient"
  },
  "Frontend": {
    "BaseUrl": "http://localhost:4200"
  }
}
```

4. Run the API

```
cd WatchmanAttendance.API
dotnet restore
dotnet run
```

- API runs at: `http://localhost:5036`
- Swagger: `http://localhost:5036/swagger`

5. Configure the frontend

- Edit `WatchmanAttendance.UI/src/app/config/app-config.ts`:
  - Set the API URL:

```
export const AppConfig = {
  apiUrl: 'http://localhost:5036/api',
  baseUrl: 'http://localhost:5036',
};
```

Add your Google Maps API key

1. Open `WatchmanAttendance.UI/src/index.html` .
2. Find or add the following line **before** the closing `</head>` tag:

```
<script src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&loading=async"></scri
```

3. Replace `YOUR_API_KEY` with your actual Google Maps API key.
4. Save the file.

> **Tip:** Restrict your API key in the Google Cloud Console to only allow requests from your production domain for security.

6. Run the frontend

```
cd WatchmanAttendance.UI
npm install
npx ng serve
```

- Frontend runs at: `http://localhost:4200`

Build

Backend

```
cd WatchmanAttendance.API
dotnet build -c Release
```

Frontend

```
cd WatchmanAttendance.UI
npm run build
```

- Output: `WatchmanAttendance.UI/dist/watchman-attendance-ui/browser/`

Deployment (Summary)

- **Backend:** Deploy to Azure App Service. Set environment variables for connection strings and JWT secrets. Update CORS in `Program.cs`.
- **Frontend:** Deploy to Cloudflare Pages or Azure Static Web Apps. Set the correct API URL in `app-config.ts`. Add SPA fallback for Angular routing.
- **Optional:** Use a Cloudflare Worker as a proxy for custom API domains.

Production Checklist

- [ ] Change default admin password and JWT secret
- [ ] Store secrets in environment variables, not source code
- [ ] Add production frontend domain to CORS
- [ ] Restrict Google Maps API key to your domain
- [ ] Deploy both frontend and backend over HTTPS
- [ ] Test QR scan, login, and attendance flow on a real device
- [ ] Ensure SPA fallback for Angular routing
- [ ] Make sure `Uploads/` folder is on persistent storage

Troubleshooting

- **QR asks for login:** Expected if not signed in. Login and flow continues.
- **Camera/GPS not working:** Both require HTTPS.
- **Angular route 404:** Add SPA fallback (`_redirects` file or platform setting).
- **API blocked by CORS:** Add frontend URL to CORS in `Program.cs`.
- **Cloudflare Worker cannot host .NET API:** Use as a proxy only.

For more details, see the full README.md.